	INSTITUTO FEDERAL DO NORTE DE MINAS GERAIS Campus Montes Claros Bacharelado em Ciência da Computação			
Disciplina: Computação Gráfica	Atividade: TP1 - Transformações 2D e Viewport	Professor: Wagner Ferreira de Barros	Valor: 10,0 pts	Nota:
Data: 10/09/2026	Alunos:			

INSTRUÇÕES

- O trabalho poderá ser realizado individualmente ou em dupla.
- O código deve ser desenvolvido em C++ usando OpenGL legacy e GLUT/freeglut. Não utilize bibliotecas de interface gráfica externas.
- Materiais consultados podem ser usados como apoio, mas devem ser citados no README. O código entregue deve ser compreendido e explicado pelos integrantes.
- Trabalhos substancialmente copiados, inclusive de colegas ou de ferramentas de IA, poderão receber anulação total ou parcial, conforme a evidência de autoria e domínio apresentada.

1. Apresentação da atividade

Neste trabalho prático, você implementará uma aplicação gráfica 2D interativa em C++ utilizando OpenGL legacy e GLUT/freeglut. A aplicação receberá uma descrição de objetos poligonais em coordenadas do sistema do mundo, aplicará transformações geométricas por matrizes homogêneas 3×3 e converterá explicitamente o resultado para uma região de viewport da janela.

O objetivo não é utilizar as funções prontas de transformação do OpenGL. A aplicação deve calcular as transformações e o mapeamento mundo $\rightarrow$ viewport, enviando ao OpenGL apenas os pontos finais, em coordenadas de pixel da janela.

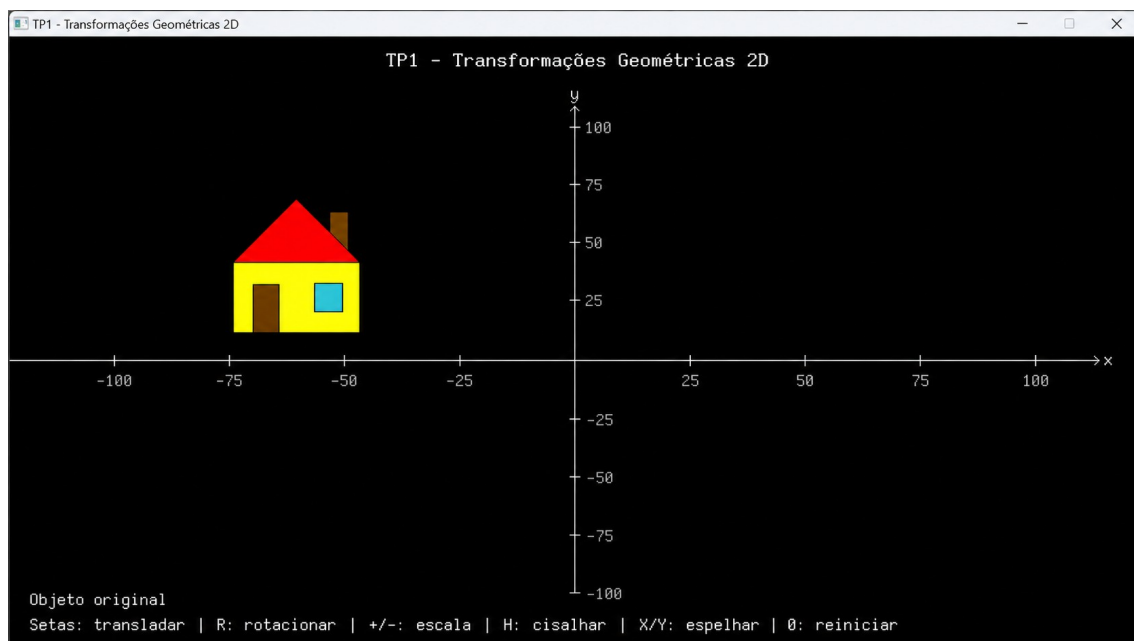


Figura 1 - Exemplo de tela inicial. A forma é definida no mundo, mas desenhada após sua conversão para a viewport.

2. Objetivos

- Representar objetos 2D como conjuntos de polígonos coloridos e vértices no sistema de coordenadas do mundo.
- Aplicar translação, rotação, escala, reflexão e cisalhamento por meio de matrizes homogêneas 3×3 .
- Compor transformações e demonstrar que sua ordem altera o resultado.
- Implementar explicitamente a transformação de janela do mundo para viewport.
- Desenvolver uma interação simples com teclado e mouse usando callbacks do GLUT.

3. Dados fornecidos e modelo da cena

Serão fornecidos arquivos iniciais contendo uma lista de objetos: casa colorida, barco colorido e moinho de vento colorido. Cada objeto é formado por um ou mais polígonos; cada polígono contém sua cor RGB e uma lista ordenada de vértices (x, y) no sistema de coordenadas do mundo. Os dados fornecidos não devem ser redefinidos em pixels nem redesenhados manualmente após cada transformação.

Os arquivos iniciais utilizam `glm::vec2` para vértices, `glm::vec3` para cores e `std::vector` para as coleções de polígonos e objetos. A aplicação deve manter, para cada objeto, uma matriz acumulada *M* inicialmente igual à identidade. Em cada redesenho, os vértices originais devem ser transformados e, somente depois, convertidos para a viewport.

$$P_mundo_original \rightarrow P_mundo_transformado = M \cdot P_mundo_original \rightarrow P_viewport$$

4. Sistemas de coordenadas e transformação de viewport

As coordenadas de todos os vértices fornecidos estarão no sistema de coordenadas do mundo. Antes de enviar qualquer vértice ao OpenGL para desenho, o programa deverá convertê-lo para o sistema de coordenadas da viewport. A viewport deve estar delimitada visualmente dentro da janela da aplicação.

Adote a convenção usual de tela: no mundo, o eixo Y cresce para cima; na viewport, o eixo Y cresce para baixo. Portanto, a implementação da transformação mundo $\rightarrow$ viewport deve considerar a inversão vertical do eixo Y. Essa inversão é necessária porque sistemas gráficos de janela e eventos de mouse normalmente adotam a origem no canto superior esquerdo, enquanto o sistema cartesiano do mundo adota a origem e o sentido positivo de Y convencionais.

As figuras deste enunciado ilustram a geometria e a interação da aplicação; os eixos nelas desenhados devem ser interpretados como referência visual do mundo. A orientação efetiva do eixo Y na viewport deve seguir a convenção definida neste item.

A avaliação verificará se o objeto continua corretamente posicionado quando a configuração da janela do mundo ou da viewport for alterada. O cálculo da transformação deve ser implementado pelos integrantes; não é necessário apresentar no relatório a fórmula de mapeamento.

5. Transformações geométricas obrigatórias

- Translação em X e Y.
- Escala uniforme e não uniforme.
- Rotação em torno da origem e em torno do centro geométrico do objeto.
- Reflexão em relação aos eixos X e Y.
- Cisalhamento em X ou em Y.
- Composição de transformações mediante atualização da matriz acumulada de cada objeto.

Para rotacionar em torno do centro geométrico (C_x , C_y), utilize a composição abaixo. O centro pode ser calculado pela média das coordenadas dos vértices do objeto.

$$M_{\text{rotação_centro}} = T(C_x, C_y) \cdot R(\theta) \cdot T(-C_x, -C_y)$$

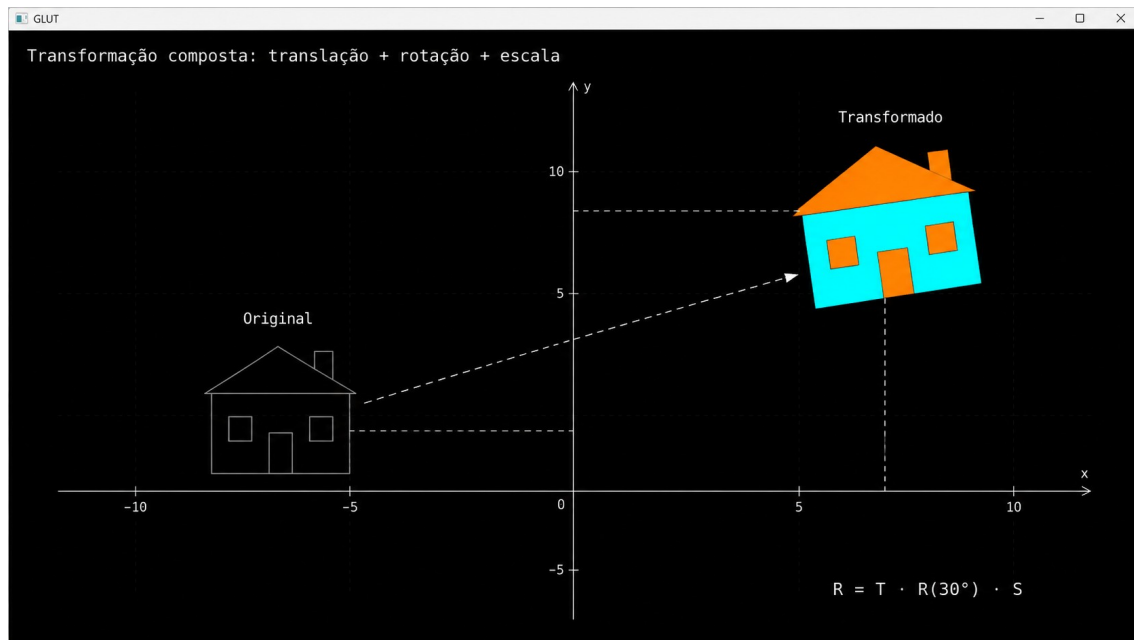


Figura 2 - Exemplo de objeto original e resultado de uma transformação composta.

6. Ordem das transformações

A multiplicação de matrizes não é comutativa. A aplicação deve oferecer uma demonstração visual de duas sequências diferentes aplicadas ao mesmo objeto: escala seguida de translação e translação seguida de escala. O resultado deve deixar clara a diferença entre $T \cdot S$ e $S \cdot T$.

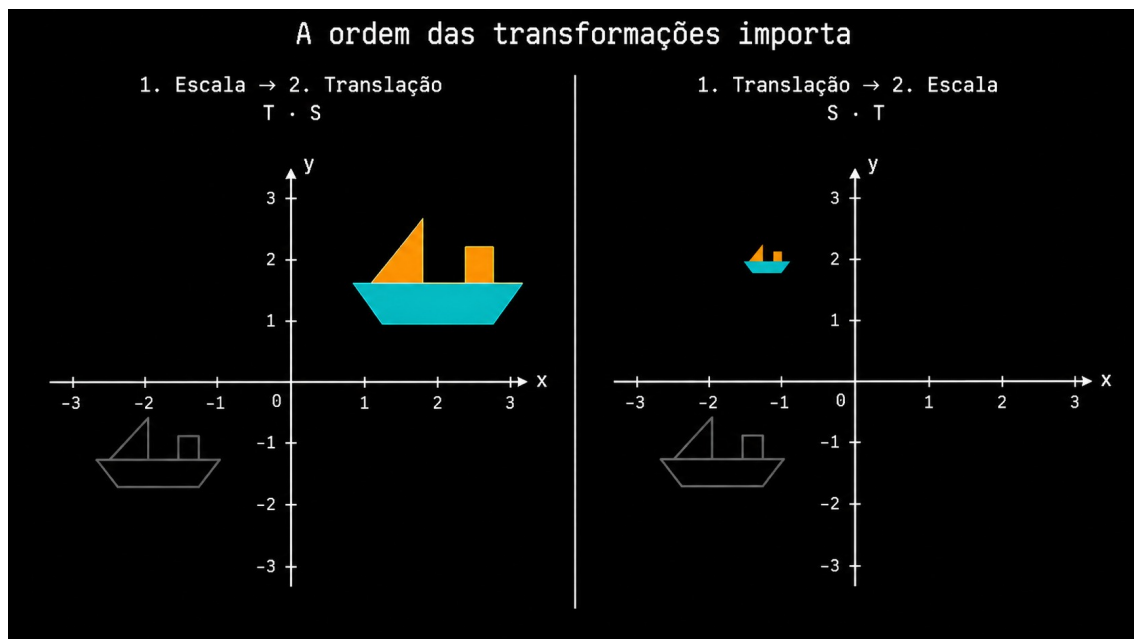


Figura 3 - Demonstração obrigatória de que a ordem das transformações altera o resultado.

7. Interação da aplicação

A aplicação deve ser interativa e utilizar somente recursos básicos de OpenGL e GLUT/freeglut. Os elementos de interface podem ser desenhados com GL_LINE_LOOP, GL_QUADS e texto produzido por glutBitmapCharacter. Não é necessário usar janelas, botões ou campos de texto de bibliotecas externas.

7.1 Painel lateral - requisito obrigatório

A tela deve conter um painel lateral, fora da viewport gráfica, para seleção do objeto ativo e configuração das transformações. Os controles podem ser acionados por clique e também possuir atalhos de teclado.

Grupo	Ação mínima	Sugestão de entrada
Seleção	Escolher um objeto da lista	Clique no nome ou tecla numérica
Translação	Alterar Δx e Δy	Setas e/ou botões direcionais
Rotação	Incrementar ou decrementar θ	R / r ou botões ↺ ↻
Escala	Alterar S_x e S_y	+ / - e/ou botões
Cisalhamento	Alterar fator H	H / h ou botões
Reflexão e reset	Espelhar e restaurar identidade	X, Y e 0

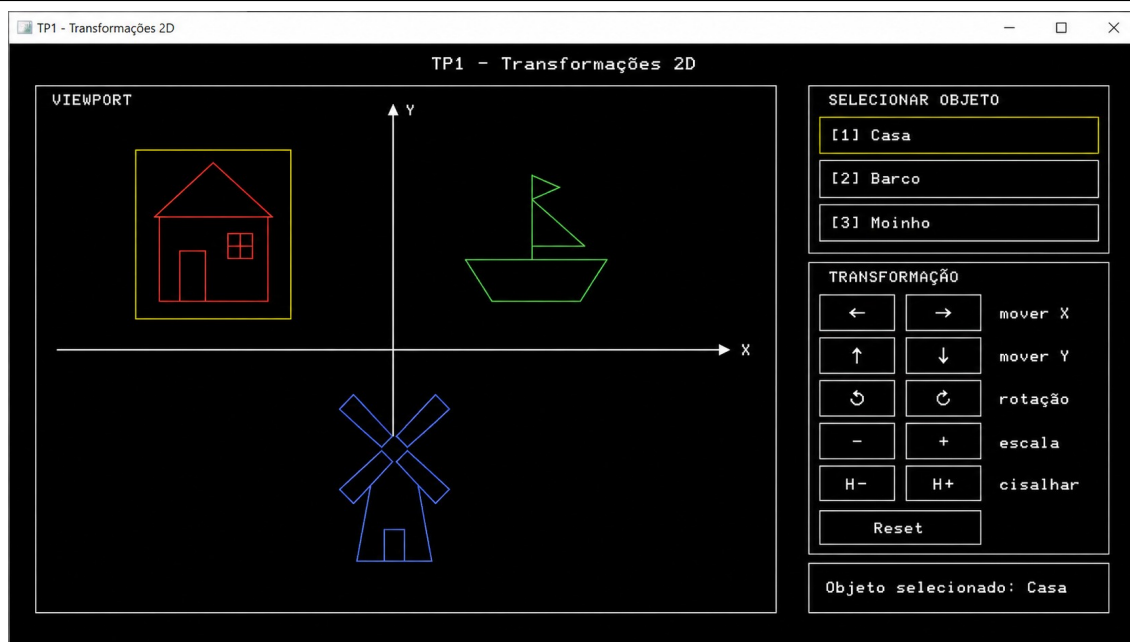


Figura 4 - Referência de painel lateral simples, desenhado manualmente com primitivas OpenGL e texto GLUT.

7.2 Seleção e arraste direto - extensão recomendada

Como extensão, implemente a seleção de um objeto por clique dentro da viewport e a translação por arraste. A posição do mouse é fornecida em pixels; para atualizar Δx e Δy , converta a posição atual e a posição anterior do mouse de viewport para mundo e use sua diferença. O objeto não deve ser movido diretamente em pixels.

mouse em pixels → conversão viewport → mundo → Δx , Δy no mundo → atualização da matriz M

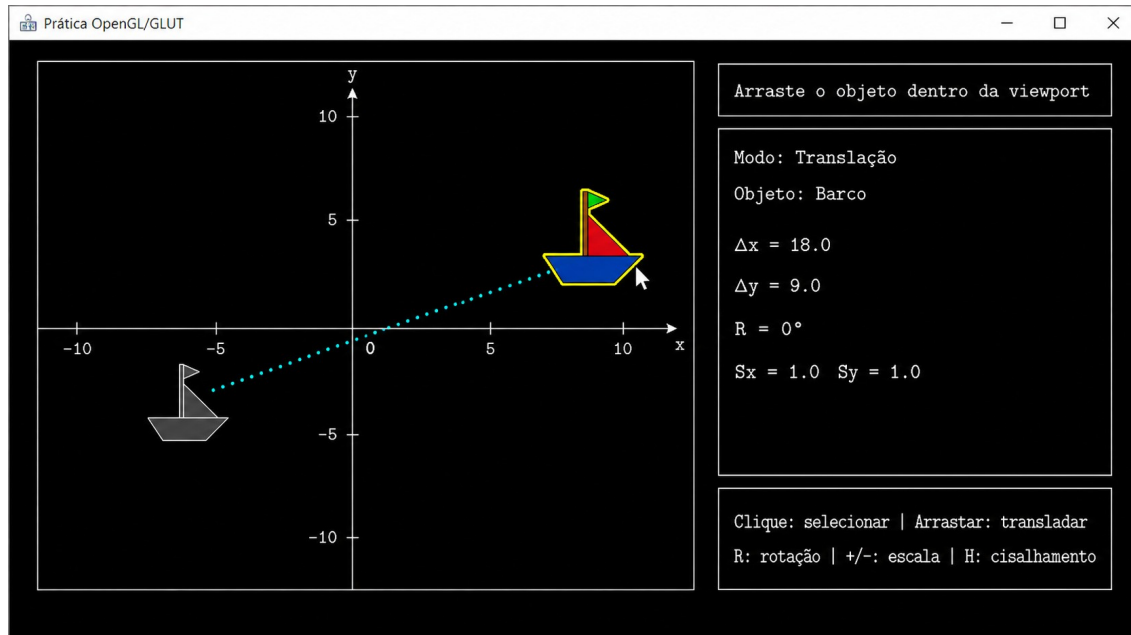


Figura 5 - Exemplo de seleção e translação opcional por arraste dentro da viewport.

8. Requisitos de programação

O projeto deve ser escrito em C++ e organizado em classes. A solução deve separar a representação geométrica, as operações matriciais, o estado da aplicação e o desenho da cena. Uma organização possível inclui as classes Vertice, Poligono, Objeto2D e Aplicacao, utilizando glm::mat3 para as matrizes; outras organizações equivalentes são aceitas desde que preservem responsabilidades claras.

- Utilizar a STL para armazenar e manipular os dados da cena: por exemplo, std::vector para coleções de objetos, polígonos e vértices.
- Utilizar obrigatoriamente a biblioteca GLM (OpenGL Mathematics) para representar e manipular vetores e matrizes. Use glm::vec2 para vértices 2D, glm::vec3 para pontos homogêneos e cores RGB e glm::mat3 para matrizes homogêneas 3×3.
- Manter os vértices originais imutáveis e associar a cada objeto sua matriz acumulada de transformação.
- Utilizar callbacks do GLUT/freeglut para desenho, redimensionamento, teclado e mouse, conforme os recursos implementados.
- Fornecer um Makefile funcional, com pelo menos os alvos all, run e clean. A compilação do projeto deverá ser realizada pelo comando make.

8.1 Uso obrigatório da GLM

A GLM é uma biblioteca header-only compatível com a convenção matemática do OpenGL. Inclua ao menos o cabeçalho abaixo nos arquivos que manipularem matrizes e vetores:

```
#include <glm/glm.hpp>
```

A GLM adota vetores-coluna. Portanto, uma transformação é aplicada multiplicando-se a matriz à esquerda pelo ponto homogêneo. O exemplo abaixo cria uma matriz de translação, representa o ponto no mundo e obtém seu resultado transformado:

```
float dx = 12.0f;
float dy = -5.0f;

glm::mat3 T(1.0f);          // matriz identidade
T[2] = glm::vec3(dx, dy, 1.0f); // terceira coluna: translação

glm::vec3 pontoMundo(x, y, 1.0f);
glm::vec3 pontoTransformado = T * pontoMundo;
```

Para compor uma nova transformação no objeto, mantenha sua matriz acumulada como `glm::mat3` e atualize-a na ordem adequada. A aplicação deve continuar a enviar à etapa de viewport apenas o ponto já transformado no mundo.

```
objeto.matrizAcumulada = T * objeto.matrizAcumulada;
```

Em Linux, a GLM pode ser instalada pelo pacote `libglm-dev`; em ambientes MSYS2/MinGW, pelo pacote `mingw-w64-x86_64-glm`. Como a GLM é header-only, normalmente não há uma biblioteca adicional a ser ligada no Makefile, apenas a garantia de que o caminho dos cabeçalhos esteja disponível ao compilador.

9. Restrições técnicas

- Não utilizar `glTranslatef`, `glRotatef`, `glScalef`, `glViewport` ou funções equivalentes como substituto das transformações geométricas e do mapeamento mundo → viewport exigidos neste trabalho.
- O OpenGL deve receber os vértices já convertidos para a viewport; o mapeamento mundo → viewport deve ser calculado pelos integrantes.
- Não utilizar bibliotecas externas de GUI, motores de jogo, imagens, texturas ou modelos prontos.
- O objeto deve ser desenhado exclusivamente por primitivas e polígonos OpenGL, a partir da lista de vértices fornecida.

10. Entrega

- Código-fonte completo, organizado em classes e compilável por meio de Makefile. O comando `make` deve gerar o executável sem modificações manuais no projeto.
- README.md ou PDF contendo: integrantes, sistema operacional utilizado, dependências, instruções de execução, comandos de interação, fontes consultadas e breve explicação da estrutura do programa.
- Duas capturas de tela: uma com transformação composta e outra com a comparação das ordens de transformação.
- Vídeo de até 2 minutos demonstrando a seleção de objetos, as transformações e a conversão para a viewport.

Prazo de entrega: 10/09/2026. A apresentação/arguição breve poderá ser solicitada para confirmar o domínio do código e das escolhas de implementação.

11. Critérios de avaliação

Critério	Pontos
Representação dos objetos, uso adequado da STL/GLM e organização da cena	1,0
Transformação explícita mundo → viewport, incluindo a inversão do eixo Y	2,0
Translação, escala e rotação com matrizes homogêneas	2,0

Reflexão, cisalhamento e rotação em torno do centro geométrico	1,5
Composição de matrizes e demonstração da ordem das transformações	1,5
Painel de seleção/controle, teclado e atualização visual	1,0
Classes, Makefile, README, capturas e demonstração	1,0
Total	10,0

12. Checklist antes da entrega

- ☐ A borda da viewport está visível e os objetos são desenhados dentro dela.
- ☐ Os vértices recebidos permanecem no sistema do mundo e são convertidos antes do desenho, com inversão correta do eixo Y.
- ☐ A matriz acumulada é mantida por objeto e pode ser reiniciada.
- ☐ A diferença entre $T \cdot S$ e $S \cdot T$ aparece claramente na aplicação.
- ☐ Os comandos do painel e/ou teclado atuam somente sobre o objeto selecionado.
- ☐ O projeto utiliza classes, STL, GLM e compila com make.

13. Referência básica

HEARN, Donald; BAKER, M. Pauline; CARITHERS, Warren. Computer Graphics with OpenGL. 4. ed. Pearson, 2014.